



Introducción y manejo de vectores en programación

Conceptos, declaración, acceso, recorrido y aplicación práctica en C++

Introducción a los vectores en programación

01

Estructuras de datos básicas en Programación I

En Programación I hemos trabajado con estructuras de datos básicas que nos permitieron resolver diversos problemas.

02

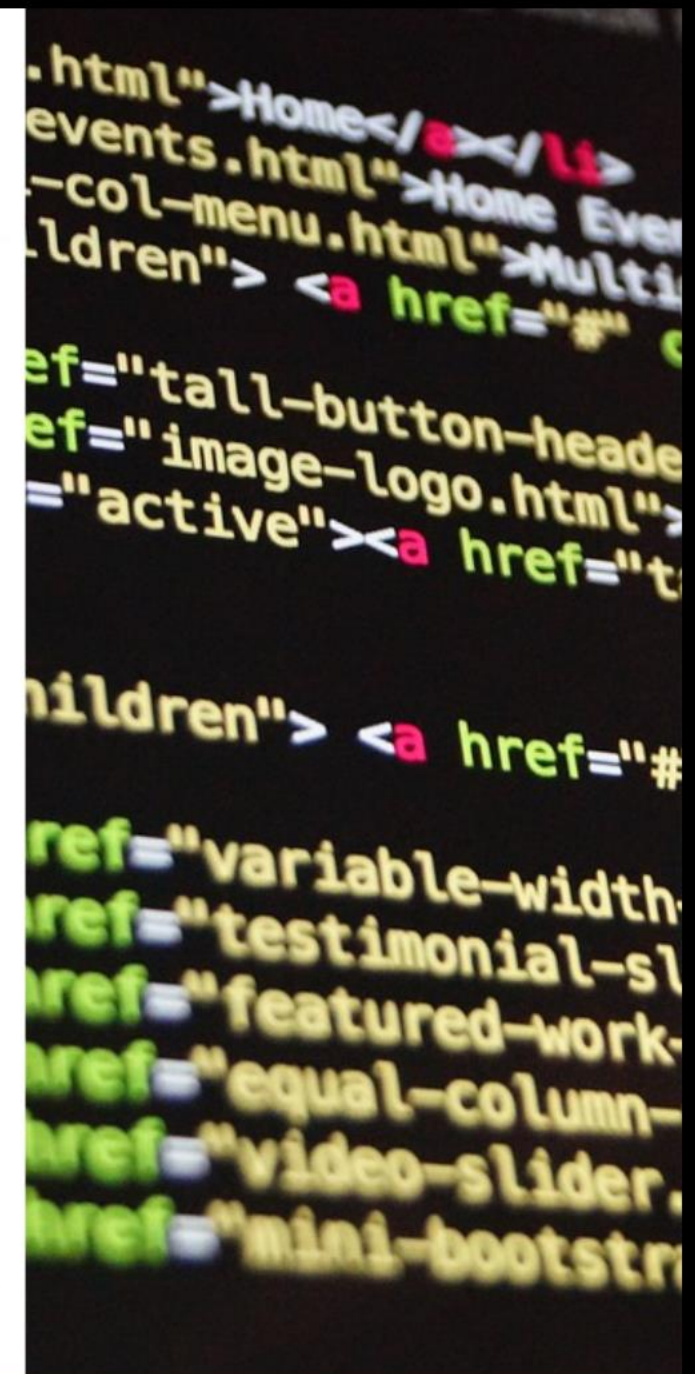
Necesidad de estructuras avanzadas

Para algoritmos más complejos necesitaremos estructuras que almacenen múltiples datos de forma práctica y sencilla.

03

Vectores: una solución fundamental

Los vectores son una solución fundamental para manejar varios elementos dentro de una misma variable, facilitando el desarrollo de programas más robustos.



¿Qué es un vector?

01

Definición de vector

Un vector es una estructura de datos que permite almacenar varios elementos en una sola variable.

02

Diferencia con una variable simple

A diferencia de una variable simple, que guarda un único dato y lo reemplaza al modificarlo, un vector mantiene múltiples valores simultáneamente.

03

Acceso y manipulación de elementos

Esto nos permite acceder y manipular cada elemento individualmente sin perder la información previa.

Representación Gráfica de Vectores vs Variables

Variable simple

- Almacena un solo valor a la vez (por ejemplo, una nota).

VS

Vector

- Almacena varios valores en una misma variable (por ejemplo, cinco notas).
- El tamaño del vector define cuántos elementos puede contener, y cada uno es accesible mediante un índice.
- Esta diferencia es clave para manejar colecciones de datos de forma eficiente.

Declarar vectores en C++

01

Declaración de vectores de tamaño fijo

En C++ declaramos vectores de tamaño fijo, definido por una constante entera.

02

Ejemplo de constante para tamaño de vector

```
const int SUCURSALES = 15;
```

03

Ejemplo de vectores en C++

```
int  
empleadosxSucursal[SUCURSALES];  
  
float recaudacion2023[12];  
  
string  
domiciliosSucursales[SUCURSALES];
```

04

Importancia del tamaño constante

El tamaño debe ser constante para evitar errores y asegurar un correcto funcionamiento.

Inicializando vectores



Declaración e inicialización de vectores

Podemos inicializar vectores al declararlos para asignar valores específicos o ponerlos en cero.



Ejemplos de inicialización

```
int legajos[5] = {1000, 2000, 3000, 4000, 5000};  
string nombres[5] = {"Bart", "Alf", "Bob", "Tony", "Marty"};  
int materiasAprobadas[5] = {}; // Inicializa todos los  
                                elementos en cero
```



Restricción de inicialización posterior

La inicialización posterior a la declaración no permite asignar valores de esta forma.

Accediendo a elementos del vector

Uso de corchetes para acceder a elementos

Utilizamos corchetes [] para acceder o modificar cada elemento individualmente.

Ejemplo de asignación de valores

```
nombresMeses[0] = "Enero";  
diasMeses[1] = 28;
```

Intercambio de valores entre elementos

```
nombresMeses[0] = nombresMeses[1];
```

Incremento de valores

```
diasMeses[0]++;
```

Mostrar valores con cout

```
cout << nombresMeses[0] << " tiene "  
<< diasMeses[0] << " días." << endl;
```

Accediendo usando Variable como Índice

```
cin >> vecPrecios[X]
```

La indexación comienza en cero

01

Convención de índices en C++

Los vectores en C++ comienzan a contar sus índices desde cero.

Esta convención es estándar en la mayoría de los lenguajes de programación.

02

Primer elemento del vector

El primer elemento está en la posición 0.

03

Último elemento del vector

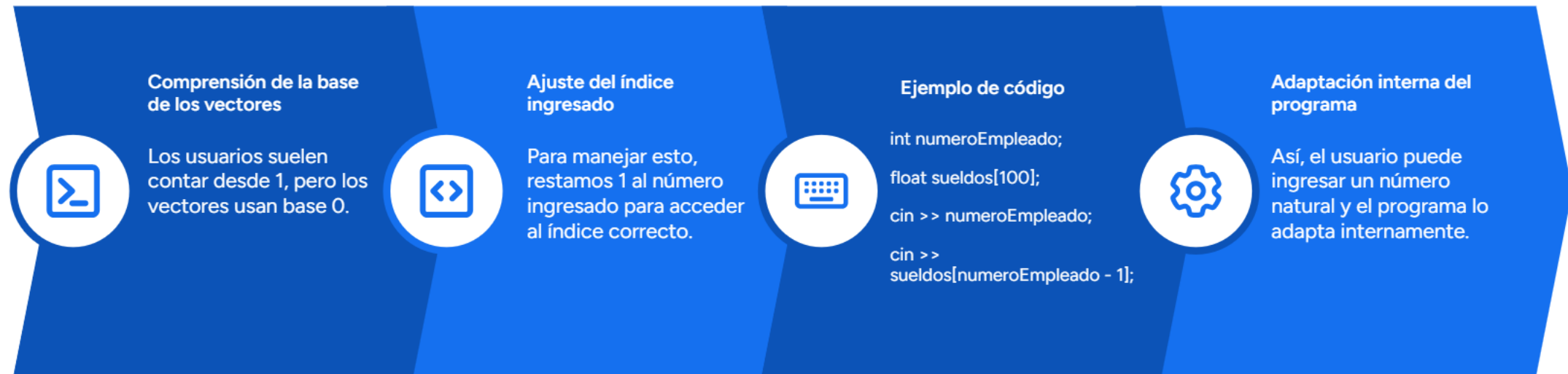
El último elemento está en la posición tamaño - 1.

04

Ejemplo práctico

Un vector de 100 sueldos tiene índices de 0 a 99.

Adaptando la entrada del usuario a la indexación vectorial



Recorriendo vectores

